

Urna Eletrônica WEB

Segurança e Auditoria de Sistemas

Equipe: Samuel De Lorenzi, Paulo Bumba e Leonardo Zonta

Disciplina: Auditoria e Segurança da Informação

Instituição: UNOESC Videira

Ano: 2026

1. Introdução: O Desafio

Apresentador: Samuel De Lorenzi

- **O Problema:** Adaptação da urna física para um ambiente 100% WEB, exigida pelo TSE para as Eleições de 2026.
- **O Desafio:** Manter o anonimato do voto enquanto se garante que a votação ocorreu em um ambiente hostil (a internet).
- **Escopo Atual:** Eleição restrita ao cargo de Presidente da República.
- **Foco do Projeto:** Garantir requisitos máximos de Segurança, Auditoria e Redundância sem perder usabilidade.

2. Arquitetura da Solução

Apresentador: Samuel De Lorenzi

- **Backend:** Desenvolvido em Python utilizando o framework **FastAPI**.
- **Banco de Dados: SQLite** (banco relacional local).
 - Por que SQLite? Simplifica o backup (é um arquivo só) e usamos o modo **WAL**¹ para garantir concorrência e integridade em quedas de energia.
- **Frontend:** Vanilla HTML, CSS e JS puros, sem frameworks pesados, garantindo carregamento rápido e redução da superfície de ataque no lado do cliente.

¹ **WAL (Write-Ahead Logging):** Técnica de banco de dados onde as mudanças são registradas em um arquivo de log seguro antes de serem aplicadas na base oficial, prevenindo corrompimento de dados.

3. Funcionalidades da Urna (Visão do Eleitor)

Apresentador: Samuel De Lorenzi

- **Autenticação:** O eleitor cria uma conta com seu CPF e uma senha forte.
- **Votação:** Interface inspirada na Urna oficial do TSE, com teclas de confirmação, correção e voto em branco.
- **Comprovante Único:** Ao votar, o eleitor recebe um hash único.
- **Verificação Restrita:** Com esse código, o eleitor pode atestar na plataforma que o sistema "recebeu e processou" seu voto, mas o código **não revela em quem ele votou**.

4. Funcionalidades de Auditoria e Administração

Apresentador: Samuel De Lorenzi

- **Rotas Restritas:** Apenas perfis designados (`admin` e `auditor`) acessam o painel de auditoria.
- **Verificação de Registros:** Confere matematicamente toda a linha do tempo do sistema em busca de adulterações no banco.
- **Boletim de Urna Eletrônico:** Gera o resultado apurado contendo uma assinatura criptográfica única para o momento do encerramento.
- **Liberação de Lotes:** O Administrador pode forçar a liberação dos votos no encerramento da eleição.

5. Modelagem e Concorrência de Dados

Apresentador: Samuel De Lorenzi

- **Separação de Identidade e Voto:** O banco possui a tabela `comparecimento` (quem votou) separada da tabela `votos` (os votos recebidos).
- **Garantia de Voto Único:** O `eleitor_id` é chave primária na tabela de `comparecimento`. O banco impede tecnicamente o duplo voto.
- **Concorrência:** Para evitar condição de corrida (dois votos ao mesmo tempo), o banco utiliza a transação serializada **BEGIN IMMEDIATE**¹.

¹ **BEGIN IMMEDIATE:** Comando SQL que cria um bloqueio de escrita instantâneo, garantindo que nenhum outro processo consiga gravar dados ao mesmo tempo, impedindo fraudes de milissegundo.

6. Autenticação e Proteção Básica

Apresentador: Paulo Bumba

- **Hashes e Salts:** CPFs não são salvos em texto puro (usamos **HMAC-SHA256**¹). Senhas são salvas com hash usando *salt* dinâmico.
- **Anti Força Bruta:** Limite estrito de 5 tentativas de login incorretas por CPF. Passou disso, a conta é bloqueada.
- **Sessão Segura (Tokens):** O acesso usa cookies restritos, eliminando vetores de ataque via JavaScript (**XSS**²).
- **Anti-CSRF:** Implementação do padrão *Double Submit Cookie* para requisições que alteram dados, evitando ações forjadas.

¹ **HMAC-SHA256:** Função matemática que mistura o CPF com uma senha secreta do servidor para esconder o dado.

² **XSS (Cross-Site Scripting):** Ataque onde scripts maliciosos injetados na página tentam roubar as credenciais ou a sessão do eleitor

7. O Sigilo do Voto: O Grande Dilema

Apresentador: Paulo Bumba

- **O Problema do Timestamp:** Na internet, se o banco registrar o momento (hora/minuto/segundo) em que a linha do voto foi criada, um administrador de TI poderia cruzar o horário de acesso do usuário no log de rede com o horário do voto no banco e quebrar o anonimato.
- **Remoção do ROWID¹:** Além do timestamp, a própria ordem física em que os dados são salvos no banco poderia delatar o eleitor.
- **Como resolvemos?** Desabilitamos o ROWID e criamos o Sistema de Lotes de Votos Pendentes.

¹ ROWID: Identificador numérico invisível que alguns bancos de dados dão a cada linha, revelando silenciosamente qual linha foi salva primeiro.

8. Criptografia e o Sistema de Lotes

Apresentador: Paulo Bumba

- Em vez de salvar o voto direto na tabela final, nós o criptografamos (com chave simétrica **Fernet**¹) na tabela `votos_pendentes`.
- Quando a urna atinge um "X" número mínimo de votos (Lote), o sistema:
 - i. Abre (decripta) todos os votos pendentes ao mesmo tempo.
 - ii. **Embaralha** todos os votos em memória.
 - iii. Salva todos no banco final com novos IDs aleatórios.
 - iv. Deleta os pendentes.
- **Resultado:** É impossível saber quem votou em quem.

¹ **Fernet:** Padrão moderno de criptografia forte onde a mesma senha é usada para embaralhar e desembaralhar a informação. É à prova de falsificações.

9. Imutabilidade e Triggers de Banco

Apresentador: Paulo Bumba

- Se um invasor acessar fisicamente o arquivo do banco de dados para alterar um voto, o banco o impedirá.
- **Triggers de Banco**¹: Criamos gatilhos nativos (`BEFORE UPDATE` e `BEFORE DELETE`) nas tabelas de `votos` e `audit_log`.
- Se um comando SQL tentar deletar ou alterar um voto existente, o banco rejeita a operação na camada física, protegendo a arquitetura de apenas-inserção (**Append-Only**²).

¹ **Triggers (Gatilhos)**: Regras programadas direto dentro do banco de dados que agem automaticamente quando uma ação (ex: Deletar) tenta acontecer.

² **Append-Only**: Arquitetura focada em segurança onde os dados só podem ser "adicionados" (Append), nunca editados ou deletados.

10. A Cadeia de Auditoria (Log Encadeado)

Apresentador: Paulo Bumba

- Criamos a tabela `audit_log` para registrar eventos críticos (logins, votos, tentativas de fraude).
- **Hash Chain (Estilo Blockchain)**¹: Cada registro de log possui um Hash gerado a partir do conteúdo do evento + **o Hash do evento anterior**.
- **Integridade**: Se o invasor tentar adulterar um registro antigo ou pular os triggers apagando o arquivo e recriando-o modificado, a quebra da cadeia matemática denuncia exatamente onde ocorreu a adulteração no painel da Auditoria.

¹ **Hash Chain / Blockchain**: Estrutura onde cada "bloco" de informação é selado usando a informação do bloco anterior, tornando impossível alterar o passado sem quebrar a corrente inteira no presente.

11. Redundância e Estratégia de Backup

Apresentador: Leonardo Zonta

- Redundância não é apenas fazer cópias, mas **garantir que a cópia não está corrompida**.
- O botão de "Fazer Backup" cria um snapshot imediato.
- **Verificação na Origem:** Durante o backup, o sistema processa `PRAGMA integrity_check`¹ para garantir que as estruturas do SQLite estão intactas.
- Gera-se um arquivo `.sha256` anexo ao backup. A restauração não acontece se a verificação falhar.

¹ `PRAGMA integrity_check`: Ferramenta nativa profunda do SQLite que varre todo o arquivo físico do banco bit a bit em busca de corrompimentos.

12. Limitações: Urna Física vs Ambiente WEB

Apresentador: Leonardo Zonta

- Apesar de toda a arquitetura desenvolvida, o ambiente WEB carrega falhas herdadas da própria internet.
- **Urna Física:** Rede isolada, hardware controlado, sem interferência externa no ato do voto.
- **Ambiente WEB:** Controle descentralizado. O eleitor usa o próprio dispositivo (frequentemente vulnerável, com vírus, ou sob coação em casa).

13. Cenários de Erro: O que pode dar errado? (Cliente)

Apresentador: Leonardo Zonta

- Casos em que nossa arquitetura não pode impedir fraudes:
 - **Phishing / Falsificação do Site:** O eleitor clica num link falso achando que é a urna e entrega seu CPF pro atacante (que votará no lugar dele).
 - **Malware (Keyloggers/Screenloggers)¹:** O celular ou PC infectado captura a tela e envia a terceiros (quebra do voto secreto).
 - **Coação Física:** O eleitor sendo obrigado a votar com uma arma na cabeça (o ambiente domiciliar não é auditável).

¹ **Keyloggers/Screenloggers:** Vírus invisíveis instalados no dispositivo da vítima que gravam tudo o que ela digita no teclado e tiram fotos da tela sem ela saber.

14. Cenários de Erro: O que pode dar errado? (Infra)

Apresentador: Leonardo Zonta

- **Comprometimento da Chave:** Se um invasor de alto privilégio vazar a chave do servidor que decripta o lote de pendentes *antes* do lote encher, e monitorar a rede, ele poderia quebrar a anonimização.
- **Ataque DDoS¹:** Embora tenhamos Limite de Taxa, grandes exércitos de máquinas infectadas (botnets) poderiam derrubar a API web no dia da eleição, impedindo eleitores de votar (Negação de Serviço).

¹ DDoS (Ataque Distribuído de Negação de Serviço): Tática cibernética onde hackers sobrecarregam um servidor disparando milhões de acessos falsos simultâneos até que ele caia.

15. Pontos a Melhorar no Futuro

Apresentador: Leonardo Zonta

- **Biometria:** Integrar login via GOV.BR com reconhecimento facial antes do voto para mitigar venda de senhas.
- **Homomorphic Encryption¹ / Zero-Knowledge Proofs²:** Em vez de confiar no nosso código para o sigilo, utilizar provas matemáticas para processar votos.
- **Descentralização:** Migrar para clusters replicados geograficamente.

¹ **Homomorphic Encryption:** Tipo de criptografia que permite somar os votos sem nunca precisar decifrá-los.

² **Zero-Knowledge Proofs:** Algoritmo que permite você provar que votou de forma válida, sem precisar revelar o seu voto para o sistema.

Apresentação do Protótipo

Apresentação prática do software em funcionamento.

Fim da Apresentação

Muito Obrigado!

Dúvidas?